9. Python Code Generation Using Structured Prompting

Design structured prompts to instruct an LLM to generate Python programs for specified computational problems and validate the generated code.

Aim

To design structured prompts for two computational problems and validate the generated Python functions using normal inputs, boundary cases, and an invalid input.

Algorithm

1. Identify each problem, its inputs, output, constraints, and function name.

2. Write structured prompts that state the role, task, input, expected output, constraints, examples, and code-only format.

3. Submit each prompt to an LLM and review the generated Python code.

4. Save the trusted factorial and GCD functions as factorial.py and gcd.py.

5. Write a validation program that imports both functions and runs the specified test cases.

6. Compare actual results or exceptions with the expected results and display PASS or FAIL.

7. Review failed cases, improve the prompt or code, and test again.

The LLM is used separately to generate the functions. The Python program in this report validates saved functions; it does not connect to an LLM API.

Structured Prompt 1: Factorial

Role: You are a Python programming assistant.
Task: Write a function named factorial(n) that calculates the factorial of n.
Input: One integer n.
Output: Return an integer.
Constraints: n is an integer; return 1 for n = 0 and raise ValueError for n < 0. Do not use external libraries or input().
Examples: factorial(0) returns 1; factorial(5) returns 120; factorial(-1) raises ValueError.
Format: Return only the complete Python function, without explanation or Markdown.

Structured Prompt 2: Greatest Common Divisor

Role: You are a Python programming assistant.
Task: Write a function named gcd(a, b) using the Euclidean algorithm.
Input: Two integers a and b.
Output: Return their non-negative greatest common divisor.
Constraints: Handle negative inputs, zeros, and gcd(0, 0) = 0. Do not use external libraries or input().
Examples: gcd(12, 18) returns 6; gcd(-8, 12) returns 4; gcd(0, 0) returns 0.
Format: Return only the complete Python function, without explanation or Markdown.

Example Generated Code

Example implementations are shown below for testing. They illustrate the expected response format and are not presented as captured outputs from an LLM run.

factorial.py

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

gcd.py

```
def gcd(a, b):
    a, b = abs(a), abs(b)
    while b:
        a, b = b, a % b
    return a
```

Save the two functions in separate files with the names shown above. Review any generated code before executing it.

Python Program: validate.py

Place this program in the same folder as factorial.py and gcd.py. It imports the functions and checks the actual values and exception against the expected results.

```
import factorial
import gcd

cases = [
    ("factorial(0)", lambda: factorial.factorial(0), 1),
    ("factorial(1)", lambda: factorial.factorial(1), 1),
    ("factorial(5)", lambda: factorial.factorial(5), 120),
    ("gcd(12, 18)", lambda: gcd.gcd(12, 18), 6),
    ("gcd(0, 0)", lambda: gcd.gcd(0, 0), 0),
    ("gcd(-8, 12)", lambda: gcd.gcd(-8, 12), 4),
]
passed = 0
for name, function, expected in cases:
    try:
        actual = function()
        ok = actual == expected
        print(name, "PASS" if ok else "FAIL",
              "| expected:", expected, "| actual:", actual)
        passed += int(ok)
    except Exception as error:
        print(name, "ERROR:", type(error).__name__)

try:
    factorial.factorial(-1)
    print("factorial(-1) FAIL | expected: ValueError")
except ValueError:
    print("factorial(-1) PASS | expected: ValueError")
    passed += 1
except Exception as error:
    print("factorial(-1) ERROR:", type(error).__name__)
print("Passed:", passed, "/", len(cases) + 1)
```

Input

Use the two Python files shown above and run validate.py. The validation program supplies these inputs automatically: 0, 1, 5, and -1 for factorial; (12, 18), (0, 0), and (-8, 12) for GCD.

Output

The image below shows the actual output obtained by running the validation program with the example implementations.

Validation program output

```
factorial(0) PASS | expected: 1 | actual: 1
factorial(1) PASS | expected: 1 | actual: 1
factorial(5) PASS | expected: 120 | actual: 120
gcd(12, 18) PASS | expected: 6 | actual: 6
gcd(0, 0) PASS | expected: 0 | actual: 0
gcd(-8, 12) PASS | expected: 4 | actual: 4
factorial(-1) PASS | expected: ValueError
Passed: 7 / 7
```

Result

Structured prompts were designed for factorial and GCD. The validation program ran seven checks on the example implementations, and all seven passed. These checks cover selected cases; they do not prove correctness for every possible input.